Rotary gated recurrent state update primitive as an Efficient Depth Substitute in Tiny Causal Language Models

Fractal Research Notes

April 16, 2026

Abstract

We report a bounded H100 proof ladder for a small recurrent control primitive, rotary gated recurrent state update primitive, inserted into a looped middle-depth causal language-model scaffold. The goal is not to claim that the primitive generally beats Transformers. The narrower question is whether the primitive provides useful depth-like computation under a fixed tiny-model experimental contract. In a 9.87M parameter model trained for 491.52M OpenLLaMA-tokenized FineWeb tokens, rotary gated recurrent control beats the strongest same-scaffold looped control without the recurrent primitive across three seeds, survives a speed-compensated control check, and remains ahead after a bounded learning-rate sweep of looped baselines. Against pure attention, rotary gated recurrent control beats 9L, 10L, and 12L attention controls on the seed-42 depth screen and falls into a three-seed tie band with 14L attention while using about 10% fewer parameters. A 16L attention model, however, beats rotary gated recurrent control on seed 42 while using more parameters, more memory, and lower throughput. These results support a modest but useful claim: in this tiny-LM regime, the rotary gated recurrent state update primitive behaves like an efficient depth substitute, but it has not yet shown a general advantage over sufficiently deeper attention. We close with a proposed 30M-50M parameter experiment needed to determine whether the effect survives beyond tiny scale.

1 Motivation

Transformer decoders remain the default architecture for language modeling because causal self-attention provides exact content-based token interaction and scales well with modern GPU kernels [8, 1]. Recurrent and state-space alternatives, including selective state-space models such as Mamba [2], offer a different tradeoff: they can provide recurrent state updates and efficient sequence processing, but they must compete with a very strong and highly optimized attention baseline.

The experiment reported here asks a narrower question than "can a recurrent primitive replace attention?" We instead test whether a recurrent primitive can act as a control mechanism for repeated middle-depth computation inside a causal decoder. The primitive is a rotary gated recurrent state update primitive. It is not inserted as a full attention replacement. It controls the injection path of a looped middle-depth scaffold, where a small set of middle attention blocks is reused for multiple recurrent passes.

The practical motivation is simple. If a small control primitive lets a model reach the quality of a deeper pure-attention model with fewer parameters or similar wall-clock cost, then it may provide a useful architectural axis for small and mid-scale language models. If the effect disappears when pure attention is tuned or deepened, then the result is still useful: it defines the boundary at which ordinary depth catches the recurrent-control scaffold.

2 Related Work and Provenance

This report is not meant to present the architecture as if it emerged in a vacuum. Several research lines shaped the design and the experimental ladder. FractalNet motivated the broader project interest in self-similar computation: deep networks can be generated by repeated expansion rules and can expose shorter and longer computational paths during training and inference [6]. The present model is not a FractalNet, but the idea that useful depth can arise from repeated structured computation is part of the intellectual lineage.

The looped scaffold is most directly informed by recent looped-language-model work. Parcae frames middle-looped models as dynamical systems over the residual stream and emphasizes stable input injection, recurrent-state control, and scaling laws for looped compute [7]. Our scaffold uses the same high-level lesson: repeated middle-depth computation needs explicit stability and injection contracts. It does not copy the Parcae architecture or claim to reproduce its training recipe.

The recurrent primitive itself is influenced by efficient sequence-mixer work. Mamba established selective state-space models as strong linear-time sequence modeling competitors [2]. Gated DeltaNet combines gating with delta-rule-style targeted memory updates and shows that recurrent/linear attention hybrids can benefit from sliding-window attention seams [9]. Kimi Linear extends this line with Kimi Delta Attention, finer-grained gating, and hardware-aware chunkwise kernels [5]. The primitive reported here borrows the broad design pressure toward gated recurrent control and efficient scan-like execution, but it is not a Gated DeltaNet, Kimi Delta Attention, or Mamba implementation.

Finally, the proof ladder is shaped by language-model scaling-law practice. Scaling-law work motivates matching parameter count, data exposure, and compute instead of over-reading a single favorable run [4, 3]. That is why this report emphasizes no-repeat token budgets, seed repeats, speed-compensated controls, and stronger attention-depth baselines.

3 Architecture

3.1 Base model

All runs use a causal decoder language model trained for next-token prediction. The base tiny model has:

- vocabulary size 32,000 using an OpenLLaMA SentencePiece tokenizer,
- model width $d = 128$,
- four attention heads,
- local context length 256,
- feed-forward multiplier 4,
- eight physical transformer blocks unless otherwise stated.

The pure attention baseline is an ordinary stack of local causal attention blocks. The Parcae family preserves the same language-model objective and tokenization, but changes the execution scaffold.

3.2 Looped middle-depth scaffold

The Parcae scaffold splits the physical transformer stack into a prelude, a middle recurrent segment, and a coda. For the default 8-layer model, the split is:

$$3 \text{ prelude blocks} \rightarrow 2 \text{ middle blocks looped twice} \rightarrow 3 \text{ coda blocks.}$$

The scaffold maintains a recurrent middle state. At each loop, it combines a bounded injection from the prelude stream with a decayed previous state, applies the middle attention blocks, and updates the recurrent state through a bounded nonlinear delta. This is a stable looped-compute scaffold, not a faithful reproduction of any external looped language-model paper.

3.3 Rotary gated recurrent state update primitive

The primitive descends from the project's earlier contractive recurrent update line, but adds two contracts that the simpler contractive update lacked: a transformed carry path before the update and an emitted output that is distinct from the latent state.

For an input token representation x_t and prior latent state s_{t-1}, the primitive computes packed input projections

$$(a_t, \theta_t, c_t, r_t) = W_{\text{in}} x_t$$

and applies a gated transformed-state update

$$u_t = \text{Rot}(W_s s_{t-1}, \theta_t),$$

$$s_t = g(a_t) \odot u_t + (1 - g(a_t)) \odot \tanh(c_t),$$

$$o_t = g(r_t) \odot s_t.$$

Here $g(\cdot)$ is the bounded gate used by the primitive implementation, W_s is the recurrent state transform, Rot applies the input-conditioned pairwise rotary transform, s_t is latent memory, and o_t is the emitted output returned to the surrounding model. The fast frozen runtime used in the earlier benchmark work exposes W_{in} as one packed projection and uses a block-diagonal recurrent state transform for the Triton scan path.

In this report, rotary gated recurrent control means that this recurrent scan is used as a control producer for the looped scaffold. It should not be read as a claim that the primitive replaces attention inside the reported model.

3.4 Control variants

We evaluate four relevant variants:

Attention-only. A standard local causal attention stack.

Parcae looped attention. The looped middle scaffold with a shared, learned scalar/channel injection.

Parcae B(x) looped attention. The same scaffold with a learned value projection and gate for the injected prelude stream.

Parcae rotary gated recurrent control. The same scaffold with a rotary gated recurrent state update primitive scan over the prelude stream. The primitive output is normalized and used to produce the injected value and gate for the middle loop.

The recurrent-control path is intentionally not a standalone sequence mixer in this experiment. It is a control primitive attached to a looped attention scaffold. This distinction matters for the claim boundary.

4 Experimental Contract

The H100 proof ladder used:

- Modal 1x NVIDIA H100 80GB HBM3,
- PyTorch 2.10.0 with CUDA 12.8,
- bf16 training,
- Triton primitive backend,
- OpenLLaMA-tokenized FineWeb CC-MAIN-2024-10 cache,
- 750,000,544 train tokens and 10,029,403 eval tokens,
- sequence length 256,
- batch size 64,
- 64 eval batches,
- deterministic seed and data-seed pairs.

Each 30,000-step lane sees

$$30{,}000 \times 64 \times 256 = 491{,}520{,}000$$

training tokens, about 65.5% of the train split. The runner enforces a no-repeat cap, so these runs do not wrap the training cache.

5 Results

5.1 Initial four-lane promotion

Table 1 shows the seed-42 promotion run. Plain looped middle-depth accounts for most of the gain over the weak 8L attention baseline. B(x) improves the looped scaffold slightly. The recurrent-control path improves it further.

Table 1: Seed-42 four-lane H100 promotion, 30k steps.

Lane	Params	Final loss	tok/s	Peak MB
8L attention-only	9.78M	5.9347	1,067,937	6,774.79
Parcae looped attention	9.78M	4.4132	878,842	6,983.55
Parcae B(x) looped attention	9.81M	4.3842	866,260	6,991.98
Parcae recurrent control	9.87M	4.3012	793,615	7,046.88

5.2 Equal-step comparison against B(x)

The cleanest same-scaffold comparison is B(x) versus the recurrent-control path. Table 2 shows the three-seed equal-step result. The recurrent control wins all three seeds.

Table 2: Equal-step finalist repeat, 30k steps.

Seed	B(x) loss	Recurrent-control loss	Delta	B(x) tok/s	Recurrent-control tok/s
42	4.3842	4.3012	-0.0830	866,260	793,615
43	4.4658	4.2972	-0.1687	908,138	832,650
44	4.3518	4.2965	-0.0553	863,253	792,434
Mean	4.4006	4.2983	-0.1023	879,217	806,233

The mean loss delta is -0.1023 in favor of the recurrent-control path, corresponding to a perplexity ratio of $\exp(-0.1023) \approx 0.9027$. In this scaffold, the primitive is not merely adding noise or capacity; it provides a repeatable control advantage over the strongest looped injection path without the recurrent primitive.

5.3 Speed-compensated B(x) pressure check

Because the recurrent-control path is slower than B(x), we also gave B(x) additional steps to spend its speed advantage. This is not an exact stopwatch-identical test, because observed H100 throughput varies by run, but it is a useful practical pressure check.

Table 3: Speed-compensated B(x) pressure check.

Seed	B(x) steps	B(x) loss	Recurrent-control loss	Delta	B(x) train min
42	32,750	4.4223	4.3012	-0.1211	9.39
43	32,725	4.5352	4.2972	-0.2380	9.45
44	32,685	4.4055	4.2965	-0.1090	10.33
Mean	–	4.4543	4.2983	-0.1560	9.72

The faster B(x) control did not catch the recurrent-control path when given extra steps. In fact, the speed-compensated B(x) runs were worse than their 30k-step counterparts at this learning rate, suggesting that simply extending the control lane at fixed LR was not enough.

5.4 Tuned looped controls without the recurrent primitive

We next gave looped controls without the recurrent primitive a bounded learning-rate sweep on seed 42: 3e-4, 6e-4, 1e-3, and 1.5e-3. The best setting was plain Parcae looped attention at LR 6e-4. It still lost to the recurrent-control path across three seeds.

The mean perplexity ratio from the mean loss delta is $\exp(-0.0772) \approx 0.9257$. The recurrent-control path therefore survives the first tuned-control rung, although it remains slower than the tuned looped control.

5.5 Pure attention depth boundary

The strongest caveat comes from deeper pure attention. We screened attention depth at the same width and context length, using LR 6e-4 for the deeper attention controls. Table 6 shows seed 42.

Table 4: Seed-42 learning-rate sweep for looped controls without the recurrent primitive.

LR	Parcae looped loss	B(x) looped loss
3e-4	4.5403	4.5320
6e-4	4.3712	4.3890
1e-3	4.4132	4.3842
1.5e-3	7.7104	5.7993

Table 5: Best tuned looped control versus recurrent control.

Seed	Tuned looped loss	Recurrent-control loss	Delta	Tuned tok/s
42	4.3712	4.3012	-0.0700	935,472
43	4.3771	4.2972	-0.0800	913,677
44	4.3781	4.2965	-0.0816	856,470
Mean	4.3755	4.2983	-0.0772	901,873

This screen shows the boundary clearly. The recurrent-control path beats the 9L, 10L, and 12L attention controls. It is essentially tied with 14L attention and loses to 16L attention on seed 42. The 16L model is larger, slower, and uses more CUDA memory, but it has better loss.

We repeated the 14L comparison across seeds because it is the boundary closest to the recurrent-control path.

The mean recurrent-control edge over 14L attention is only -0.0039 loss, a perplexity ratio of about 0.9961. This should be interpreted as a tie band, not a decisive quality win. The practical result is nevertheless meaningful: the recurrent-control path reaches the 14L attention boundary with about 10% fewer parameters and similar throughput.

5.6 Speed-spend pressure against 16L attention

We also let the recurrent-control path spend some of its speed advantage against the 16L attention model. A 33k-step run reached 4.2835 loss, and a 34k-step run reached 4.2812 loss. Both narrowed the gap but did not catch the 16L attention result of 4.2758 on seed 42.

6 Discussion

The core result is best described as an efficient-depth signal. The recurrent-control scaffold does not simply beat all attention. It also does not collapse once attention is strengthened. Instead, it occupies a useful middle ground:

- It reliably beats same-scaffold controls without the recurrent primitive.
- It beats parameter-near and compute-near attention controls.
- It sits in a tie band with a much deeper 14L attention model.
- It loses to 16L attention at this context length and token budget.

Table 6: Pure attention depth screen, seed 42.

Lane	Params	Final loss	tok/s	Train min
8L attention, LR 1e-3	9.78M	5.9347	1,067,937	7.67
9L attention, LR 6e-4	9.98M	4.4459	1,091,606	7.50
10L attention, LR 6e-4	10.17M	4.3649	981,974	8.34
12L attention, LR 6e-4	10.57M	4.3304	897,467	9.13
14L attention, LR 6e-4	10.97M	4.3044	773,922	10.59
16L attention, LR 6e-4	11.36M	4.2758	721,719	11.35
Recurrent control, LR 1e-3	9.87M	4.3012	793,615	10.32

Table 7: 14L attention boundary repeat.

Seed	14L loss	Recurrent-control loss	Delta	14L tok/s	Recurrent-control tok/s
42	4.3044	4.3012	-0.0032	773,922	793,615
43	4.2960	4.2972	+0.0011	772,807	832,650
44	4.3061	4.2965	-0.0097	847,395	792,434
Mean	4.3022	4.2983	-0.0039	798,041	806,233

This is the first result in this line of work that looks like a credible architectural role rather than a primitive leaderboard curiosity. The primitive is not replacing attention. It is controlling repeated middle-depth computation and acting as a compact substitute for some ordinary depth.

There are two immediate interpretations. The optimistic interpretation is that rotary gated recurrent control supplies a recurrent control state that helps the looped middle blocks refine hidden representations more efficiently than adding several ordinary attention layers. The skeptical interpretation is that looped middle depth itself provides most of the gain, and the recurrent-control path is a small regularizer or gating improvement whose advantage may vanish at larger scale or longer training. The current data favors the first interpretation within the tiny contract, but it cannot settle the scaling question.

7 Limitations

The result should not be overstated.

- The models are tiny: roughly 10M parameters.
- Context length is only 256 tokens.
- The attention baselines were tuned only lightly.
- The strongest pure attention model tested, 16L, beat the recurrent-control path.
- We do not yet have a 30M-50M scale result.
- We do not yet have long-context evidence.
- We have not shown generation quality, downstream task quality, or quantization behavior for this specific scaffold.

The supported claim is therefore:

> In the current 10M-parameter, 256-context benchmark, the rotary gated recurrent state update primitive acts as an efficient depth substitute inside a looped attention scaffold. It beats same-scaffold controls and reaches the 14L pure-attention boundary with fewer parameters, but it does not beat sufficiently deeper attention.

8 Next Experiment

To determine whether this is more than a tiny-model artifact, the next experiment should move to 30M-50M parameters. A practical first target is:

- width $d = 320$,
- 16 physical layers for the recurrent-control scaffold,
- 8 heads,
- roughly 40.8M parameters,
- 1.0B to 1.25B training tokens,
- fixed OpenLLaMA tokenizer and FineWeb-derived train/eval split.

The control set should include:

- the scaled recurrent-control scaffold,
- a parameter-near attention baseline,
- a stronger depth-near attention baseline,
- optionally a looped control without the recurrent primitive if budget allows.

One H100 is sufficient for this next rung. Multi-GPU training is not required because the models fit comfortably on one H100 and the current harness is single-GPU. If parallel compute is available, it should be used for independent lanes or seeds rather than distributed training.

A bounded compute request of 50-100 H100-hours would cover:

- token-cache preparation and smoke tests,
- one-seed 30M-50M screening,
- finalist promotion to a larger token budget,
- seed repeats only if the first screen remains alive.

9 Conclusion

Rotary gated recurrent state update primitive is not yet a transformer replacement result. It is a credible efficient-depth result. In a controlled tiny-LM H100 ladder, the recurrent-control path improves a looped middle-depth attention scaffold beyond matched controls without the recurrent primitive, reaches the 14L attention boundary with fewer parameters, and remains competitive until pure attention is made substantially deeper. The next scientific question is whether this role survives at 30M-50M parameters and longer context. If it does, the primitive deserves a more serious scaling-law study. If it does not, the current work still clarifies where recurrent control helps and where ordinary attention depth catches up.

References

[1] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. Flashattention: Fast and memory-efficient exact attention with io-awareness. *Advances in Neural Information Processing Systems*, 2022.

[2] Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces. *arXiv preprint arXiv:2312.00752*, 2023.

[3] Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, Tom Hennigan, Eric Noland, Katie Millican, George van den Driessche, Bogdan Damoc, Aurelia Guy, Simon Osindero, Karen Simonyan, Erich Elsen, Jack W. Rae, Oriol Vinyals, and Laurent Sifre. Training compute-optimal large language models. *arXiv preprint arXiv:2203.15556*, 2022.

[4] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*, 2020.

[5] Kimi Team. Kimi linear: An expressive, efficient attention architecture. *arXiv preprint arXiv:2510.26692*, 2025.

[6] Gustav Larsson, Michael Maire, and Gregory Shakhnarovich. Fractalnet: Ultra-deep neural networks without residuals. In *International Conference on Learning Representations*, 2017.

[7] Hayden Prairie, Zachary Novack, Taylor Berg-Kirkpatrick, and Daniel Y. Fu. Parcae: Scaling laws for stable looped language models. *arXiv preprint arXiv:2604.12946*, 2026.

[8] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, 2017.

[9] Songlin Yang, Jan Kautz, and Ali Hatamizadeh. Gated delta networks: Improving mamba2 with delta rule. In *International Conference on Learning Representations*, 2025.